

INSTRUCTIONS

1. Write answers in the separate ANSWER SHEET provided to you.
2. Write name and RollNo. in the space provided below and also on the answer sheet.
3. No questions will be entertained during the exam.
4. At the end of examination, return the question paper and answer sheet.

RollNo

--	--	--	--	--	--	--	--	--

Name:

--

SECTION-II

Write the answer in the space corresponding to the question in the answer sheet. Wherever not specifically mentioned, the answer could be the output of the program or compile time error or run-time error as per your judgement.

Question-1 to Question-10 carry 2 points each.

Question-11 to Question-19 carry 4 points each.

Question-20 to Question-22 carry 8 points each.

1. #include <stdio.h>

```
int* foo(int *);
```

```
void main() {  
    int a[5] = {1,2,3,4,5};  
    int b[5], *c;  
    b = foo(a);  
}
```

```
int* foo(int x[]) {  
    int i;  
    for(i=0; i<5; i++)  
    {  
        x[i] += x[i];  
        printf("%d, ", x[i]);  
    }  
    return x;  
}
```

2. #include <stdio.h>

```
int add(int x, int y) {  
    int s;  
    s = x+y;  
    return;  
}
```

```
void main() {  
    int a,b, sum=0;  
    a=5; b=7;  
  
    sum = add(a,b);  
    printf("Sum = %d", sum);  
}
```

3.

```
#include <stdio.h>  
int func(int *x, int y) {  
    if(y % 2) {  
        *x = 1;  
        return 1;  
    }  
    else {  
        *x = 0;  
        return 0;  
    }  
}  
void main() {  
    int a=8, b=5, c=9;  
  
    c = func(&a, b);  
    printf("a=%d, b=%d, c=%d", a,b,c);  
}
```

4. #include <stdio.h>

```
int mult(int x, int y)  
{  
    int m;  
    m = x * y;  
    return (m, x, y);  
}
```

```
void main()  
{  
    int a,b, prod=0;  
    a=5; b=7;  
  
    prod = mult(a,b);  
    printf("Prod = %d", prod);  
}
```

5. Fill in the blanks so that the program gives correct output. Write the answer in the answer sheet. Here l, b, h, v are the variables inside the structure.

```
#include <stdio.h>

struct xyz{
    -----
    -----
} -----

void main()
{
    a.l = 5;
    a.b = 6;
    a.h = 4;
    a.v = 456.3;
    printf("Information is
           %d, %d, %d, %f\n",
           a.l, a.b, a.h, a.v);
}
```

6. How does the OS handle input-output?
7. List any four states of a process.
8. What is the unit that translates logical address to physical memory address called?
9. Is it necessary to load the complete program in the main memory before starting execution? Yes/No. Give reason.
10. Write the output of the program.

```
#include<stdio.h>

struct s_car{
    char make[32];
    int price;
    int year;
}S1;

union u_car{
    char make[32];
    int price;
    int year;
}U1;

void main() {
    printf("size of struct car =%ld ",
           sizeof(S1));
    printf("size of union car =%ld ",
           sizeof(U1));
}
```

11. Write the output.

```
#include <stdio.h>
void foo(char *);

void main()
{
    char s[] = "CS101 End-Sem";
    foo(s);
}

void foo(char *a)
{
    char c;
    c = a[0];

    if(c == '\0')
        return;
    foo(a+1);
    printf("%c", c);
}
```

12. Fill in the blanks and also write the output of the completed program. Write your answer statements in the answer sheet.

```
#include <stdio.h>

int countdigits(int n)
{
    -----

    if(n == 0 ) return 1;

    c++;
    countdigits(-----);

    return c;
}

void main()
{
    int x = 346287, cnt=0;
    cnt = countdigits(x);
    printf("Count = %d", cnt);
}
```

13. #include <stdio.h>

```
int a=0;

void fun(int c) {
    int b;
    c = 6; b =5;
    printf("a=%d, b=%d, c=%d\n", a,b,c);
    c++;
}

---> <contd. on next page> ..
```

```

void main()
{
    int a,b,c;
    a = 1; b=2; c=3;
    printf("a=%d, b=%d, c=%d\n", a,b,c);
    {
        a=3;
        printf("a=%d, b=%d, c=%d\n", a,b,c);
    }

    fun(a);
    a++;
    printf("a=%d, b=%d, c=%d\n", a,b,c);
}

```

14. The following program goes in an infinite loop. Identify the error and suggest correction.

```

#include <stdio.h>

int display()
{
    int i=0;
    i++;
    printf( "%d, ", i);
    if( i == 50)
        return 0;
    else
        display();
}

void main()
{
    // this program prints the first 50 numbers
    display();
}

```

15. While implementing a stack using a singly-linked-list, would you prefer to use the "head-node" as the top of the stack or the "tail-node" as the top? Give your choice and justify in one sentence.
16. For the given code snippet, write the remaining code in the `main()` function to assign values to the structure variable using `*gps`. Use `scanf` to read and assign the values. Write answer in the answer sheet.

```

struct location{
    float latitude;
    float longitude;
    char  continent[10];
};

void main()
{
    struct location *gps;

    -----
    .....
    -----
}

```

17. What happens when the CPU switches from one process to another? What is the event called?
18. Write the equivalent assembly language statements to do the following:
`Sum = var1 + var2;`
19. Identify the entries of the mapping table for the logical and physical memory. Write the mapping pairs in the answer sheet.

Logical addr	Data
1	A
2	B
3	C
4	D
5	E
6	F

Physical addr	Data
1	
2	B
3	
4	A
5	
6	
7	E
8	
9	D
10	

20. Complete the definition of the function `add_marks` and complete the C-statement to call the function from the main routine.

```

#include <stdio.h>

struct marks{
    int subject1;
    int subject2;
};

void add_marks(_____)
{
    //add marks for subject1 for both exams
    //and store in subject1 of total.

    -----

    //add marks for subject1 for both exams
    //and store in subject1 of total.

    -----
}

void main()
{
    struct marks exam1, exam2, total;

    exam1.subject1 = 10;
    exam1.subject2 = 8;

    exam2.subject1 = 9;
    exam2.subject2 = 6;

    add_marks(____);
    printf("Final total marks = %d, %d",
        total.subject1, total.subject2);
}

```

21. Write the code to push an element on the stack (implemented as a linked list). The struct is given and the function template shown. **Write the code for the function in the answer sheet.**

```
typedef struct{
    int data;
    struct entry *next;
}stack;

void push(stack **top, int a)
{
    stack *tmp;

    -----
    ...
    -----
}
```

22. Below is the code written for 3 modules in different files. Address for object modules are logical addresses.

WRITE The Answer on this page in the blank table on the right.

Object module A

Address	Code
399	
...	
300	CALL B
...	
200	CALL C
...	
0	GOTO 200

Object module B

Address	Code
599	
...	
500	CALL C
...	
300	MOV Q to X
...	
0	GOTO 300

Object module C

Address	Code
499	
...	
400	MOV S to X
...	
200	MOV P to Y
...	
0	GOTO 200

What will be the output of the Linker when the modules are loaded in the memory as shown in the next table. The address in the table represents physical address.

For this question write the final answer in a similar format in the adjacent blank table below.

Object modules in memory BEFORE Linking

Address	Code	Module
1599		Object module B
...		
1500	CALL C	
...		
1300	MOV Q to X	
...		Object module C
1000	GOTO 300	
...		
900	MOV S to X	
...		Object module A
700	MOV P to Y	
...		
500	GOTO 200	
...		Object module A
400	CALL B	
...		
300	CALL C	
...		
100	GOTO 200	
...		
0		

Object modules in memory AFTER Linking

Address	Code	Module
1599		Object module B
...		
1500		
...		
1300		
...		Object module C
1000		
...		
900		
...		Object module A
700		
...		
500		
...		Object module A
400		
...		
300		
...		
100		
...		
0		